

D3-Demo Path B Backend System Report

Scan date: 2026-04-16

Scope: Full backend (runtime core + V2 frozen core + Research Core sidecar + tests)

Basis: Authoritative docs (D3 Proposal / Roadmap / Path C Vision / D3_BOUNDARY.md), legacy V2 materials (SPEC.md, SPEC_BRIDGE.md, legacy_v2/), and direct source inspection of 29 Python modules (~10.2k LOC) + 31 test files (1040 tests).

Note on a referenced document: The prompt lists `Supply_Chain_Twin_V2_系统总览报告.docx` as a V2 baseline — this file is **not present** in the workspace (searched root and `legacy_v2/`). V2 material I actually used: `SPEC.md` (28KB), `SPEC_BRIDGE.md` (24KB), and `legacy_v2/` (6 files including `V2_UPGRADE_PLAN.md`, `WEEK2_HANDOFF.md`, `project_handover.md`).

1. Executive Summary

- **One-line identity:** D3-Demo is a synthetic, deterministic, event-driven semi-autonomous exception-handling loop for supply-chain disruption response, built as an outer orchestrator around a mature V2 reasoning stack.
 - **Current delivery boundary:** Phase 0 (contracts) → Phase 1 (policy gate) → Phase 2 (event engine + hot-patch) → Phase 3 (execution adapters + outcome feedback) → Phase 3.5 (core hardening) are **all complete and test-green**. Phase 4 (event-centric UI) and Path C (learning / adaptive / replan / correlation) are **not started**.
 - **Strongest capability:** A deterministic, policy-gated, closed-loop simulator that can consume a synthetic event stream, classify, reason, route, execute simulated actions, and inject statistical outcome summaries into subsequent reasoning — all while keeping the Research Core oracle-truth sidecar cleanly isolated. 1040/1040 tests pass.
 - **Biggest limit:** No human-facing surface beyond the V2 Streamlit case viewer; no real UI for the event loop; outcome summary is piped into `scenario_context` but agents do not yet *consume* it in any differential way (plumbing is built, behavior change isn't).
-

2. System Identity and Boundary

What D3-Demo is now (per proposal §3 + actual code):

- An event-driven exception-loop **demonstrator** operating on a bounded 2×2×2×2 synthetic network (2 suppliers, 2 warehouses, 2 carriers, 2 customer zones).

- A **closed loop**: event → state patch → reasoning slice → policy gate → execution adapter → outcome store → summary injected into next event's scenario context.
- A **deterministic demo**: all randomness removed; timestamps fixed; severity→risk mapping static; action picks governed by sorted-id enumeration.

What it explicitly is not:

- Not a production ERP/WMS/TMS integrator (no external adapters, no real APIs touched).
- Not enterprise-grade execution OS (no auth, no multi-tenancy, no persistence).
- Not a thesis-first oracle-replay system (that's the Research Core sidecar role).
- Not a general control tower (bounded 4-action set, single active order at a time).

Path B closed loop (what the system currently does per event):

event → apply_event_patch → scenario_context + outcome_summary injection → ops/cost/governance nodes → policy_gate → {AUTO_EXECUTE: adapter→outcome_store, HUMAN_REQUIRED: skip, flagged for review}

V2's role in the D3 system:

- **Reusable asset source**: `twin_state.py`, `agents/*` (operations/cost/governance), `rag_setup.py`, `retrieval.py`, `llm_backend.py`, `supervisor.py` reused wholesale as inner reasoning slice members.
- **Baseline**: 20 oracle cases (M01-14, P01-06) remain as regression references.
- **Research Core sidecar**: `evaluation.py` + `action_code_mapper.py` + `data/cases/*.json` — live, tested, but **never reachable from the D3 event-driven runtime path**.

Research Core sidecar role: A frozen evaluation pipeline that runs on oracle-backed cases through the `legacy graph.compile_graph()` path (Streamlit `app.py` still uses it). Computes regret / override metrics. Isolated by import discipline: only `graph.py::evaluation_node` and `action_code_mapper.py` import from `evaluation.py`; neither is on the D3 runtime path.

Why not yet a product system: no execution safety rails outside the synthetic net, no real outcome physics, no human-facing workspace, summary injection exists but agents don't react to it differentially, no persistence, no schema evolution path, no observability tooling.

3. Repo Structure Map (module clusters)

Cluster	Files	Role
D3 Event-Driven Runtime Core (new)	src/event_schema.py (114), src/event_engine.py (249), src/event_state_mapper.py (137), src/event_loop.py (399), src/policy_gate.py (184), src/execution_adapters.py (526), src/outcome_schema.py (98), src/outcome_store.py (159)	Phase 0-3 additions. Owns all event-driven behavior.
D3 Bridge / Extended (additive)	src/twin_state.py (864, + apply_event_patch / + apply_action_outcome), src/graph.py (539, + policy_gate_node / + auto_approve_shim / +opt-in policy routing)	V2 files extended additively; V2 semantics preserved.
V2 Frozen Reasoning Core	src/agents/operations_agent.py (1013), src/agents/cost_agent.py (476), src/agents/governance_agent.py (930), src/supervisor.py (345), src/rag_setup.py (297), src/retrieval.py (1820), src/llm_backend.py (481), src/llm_providers/* (235)	Reused as-is. Never modified by D3.
Research Core / Evaluation Sidecar	src/evaluation.py (328), src/action_code_mapper.py (205), data/cases/*.json (20), data/mappings/case_to_scenario_map.json , scripts/phase6_batch_eval.py , scripts/batch_eval_runner.py	Oracle-truth evaluation pipeline. Only reachable from legacy graph, never from D3 runtime path.
Legacy Bridge / Case Replay	src/event_sequence.py (84, Phase 1 fallback runner — case_id list → legacy graph with policy gate enabled), src/case_translation.py (161)	Phase 1 Path-A fallback. Not on D3 main runtime path.
Access Layer (V2)	app.py (root, Streamlit, 644 lines), src/api/app.py (FastAPI, 263), src/api/runner.py , src/api/schemas.py	Streamlit is active demo surface (V2 case viewer + D3 policy-gate pill); FastAPI is frozen.
Test Layer	31 files, 1040 tests	V2 regression + D3 per-phase + Phase 3.5 hardening.
Governance / Docs	D3_Demo_Project_Proposal.docx , D3_Demo_Implementation_Roadmap.docx , D3_Demo_Path_C_Vision.docx , D3_BOUNDARY.md , SPEC.md , SPEC_BRIDGE.md , legacy_v2/*	Authority + V2 history.
Data	data/configs/baseline_network.json (D3 runtime seed), data/scenarios/*.json (7 V2 scenarios), data/knowledge/*.md (5 RAG docs), data/cases/*.json (20 oracle cases), data/phase6_eval/* (V2 batch output), data/baseline_twin_state.json	Mix of runtime seed (1 file) + frozen research material + V2 artifacts.
Scripts	scripts/batch_eval_runner.py , scripts/phase6_batch_eval.py , scripts/build_cases_from_bank.py , scripts/retrieval_benchmark.py	V2 batch tooling. Not part of D3 runtime path.

4. Runtime Architecture

Two parallel paths, shared reasoning sub-graph

D3 runtime path (Phase 2/3)

EventPayload → event_state_mapper.map_event_to_state (pure)



TwinState.apply_event_patch (atomic _run_patches)



scenario_context + outcome_summary (sideband injection, NOT in GovernanceOutput)



inner reasoning slice (direct node function calls, NOT compiled graph)

operations_agent_node → cost_agent_node → governance_agent_node
→ policy_gate_node



AUTO_EXECUTE

execute_action

→ ExecutionOutcome

→ outcome_store.append

→ live state = new_state

HUMAN_REQUIRED

(default: awaiting_human_review)

(auto_approve_escalations=True → demo override)

(record_routing, no execution)

Legacy / Research-Core path (V2, still live via app.py)

```

case_id → graph.compile_graph().invoke({case_id, supervisor_instruction,
                                         policy_gate_enabled?})
      |
load_case_or_scenario → inject_scenario
      |
operations → cost → governance → policy_gate
      |
      AUTO_EXECUTE → auto_approve_shim (synthetic APPROVE supervisor_output)
      HUMAN_REQUIRED → supervisor (real APPROVE/VERIFY/OVERRIDE)
      |
evaluation_node (reads oracle case JSON, computes regret / override metrics)
      |
      END

```

D3 runtime path (Phase 2/3)

```

EventPayload → event_state_mapper.map_event_to_state (pure)
      |
      ▼
TwinState.apply_event_patch (atomic _run_patches)
      |
      ▼
scenario_context + outcome_summary (sideband injection, NOT in GovernanceOutput)
      |
      ▼
inner reasoning slice (direct node function calls, NOT compiled graph)
  operations_agent_node → cost_agent_node → governance_agent_node
  → policy_gate_node

```

AUTO_EXECUTE execute_action → ExecutionOutcome → outcome_store.append → live state = new_state	HUMAN_REQUIRED (default: awaiting_human_review) (auto_approve_escalations=True → demo override) (record_routing, no execution)
--	---

Legacy / Research-Core path (V2, still live via app.py)

```

case_id → graph.compile_graph().invoke({case_id, supervisor_instruction,
                                         policy_gate_enabled?})

      |
load_case_or_scenario → inject_scenario
      |
operations → cost → governance → policy_gate
      |
    AUTO_EXECUTE → auto_approve_shim (synthetic APPROVE supervisor_output)
    HUMAN_REQUIRED → supervisor (real APPROVE/VERIFY/OVERRIDE)
      |
evaluation_node (reads oracle case JSON, computes regret / override metrics)
      |
      END
  
```

Critical boundary: inner vs outer

Inner reasoning slice (`_run_reasoning_slice` in `event_loop.py:56-101`): Directly calls V2 node functions `operations_agent_node` / `cost_agent_node` / `governance_agent_node` / `policy_gate_node`. Builds a minimal `GraphState`-compatible dict, sets `policy_gate_enabled: True`, runs them sequentially.

Does not invoke `compile_graph()`. Does not touch `load_case_or_scenario` / `supervisor` / `evaluation_node` / `auto_approve_shim`. This is the "outer orchestrator reuses inner reasoning sub-graph" pattern enforced by Phase 2 roadmap.

Outer orchestrator (`run_event_loop` in `event_loop.py:116-234`): Owns the event iteration, state mutation, routing decision, execution dispatch, outcome appending, and summary injection. **Does not** modify or replace the `LangGraph` graph.

Modules on the D3 main runtime path (actually touched per event)

`event_schema.EventPayload` → `event_state_mapper.map_event_to_state` → `twin_state.apply_event_patch` → `outcome_store.summary` (sideband)
→ `graph.{operations_agent_node,cost_agent_node,governance_agent_node,policy_gate_node}` → `execution_adapters.execute_action` →
`twin_state.apply_action` + `compute_total_cost` → `outcome_store.append` / `record_routing` → `event_loop._build_twin_summary`.

Modules NOT on D3 main path (but still alive)

- `graph.load_case_or_scenario`, `graph.supervisor_node`, `graph.evaluation_node`, `graph.auto_approve_shim`: Only reachable via the compiled graph (used by `app.py` + `src/api` + `event_sequence.py` Phase-1 fallback).
 - `evaluation.py`, `action_code_mapper.py`: Only reachable via `graph.evaluation_node`.
 - `case_translation.py`: V2 utility, not referenced from D3 runtime.
 - `scripts/*.py`: Offline V2 batch tooling.
 - `src/api/*`: FastAPI frozen; no longer primary surface.
-

5. Key Modules Deep Dive

`src/twin_state.py` (864 lines) — Hybrid core

Responsibility: Pydantic-validated runtime state (suppliers / warehouses / carriers / customer_zones / active_order / cost_policy / current_disruptions) + three mutation APIs.

I/O:

- `initialize_from_config(path)` → `TwinState` from JSON
- `inject_shock(shock_config)` — V2 atomic entity-patch (via `_run_patches`)
- `apply_event_patch(event: EventPayload)` — D3 Phase 2 atomic mutation mapping the 6 event types to patch lists
- `apply_action(action: Action)` — V2 execution mutation (EXPEDITE/TRANSFER/COMPENSATE/NO_ACTION)
- `compute_total_cost()` — deterministic runtime cost computation
- `apply_action_outcome(outcome: ExecutionOutcome)` — D3 Phase 3 narrow replay utility
- `to_prompt_context(...)` — SPEC §8 stable JSON serialization for agents

Touched layers: Operational only. Never emits AI/ALT1/ALT2; never emits APPROVE/VERIFY/OVERRIDE.

Key contracts:

- All three atomic mutation paths go through `_run_patches` or `apply_action`; no fourth mutation style.

- Validators enforce non-negative inventory, demand, ETA, capacity; cost policy positivity; `reliability_score` $\in [0,1]$.
- MVP cardinality fixed at $2 \times 2 \times 2 \times 2$ (validated in `initialize_from_config`).

Risk points: The atomic draft-copy pattern doubles memory per mutation (acceptable at current scale). `apply_action_outcome` relies on adapter-produced `state_delta` using `set`-only patches for idempotency; if ever an `add/multiply` patch leaks in, double-apply would diverge.

Maturity: Very high. 25 tests in `test_twin_state_costs.py` + coverage across other suites.

`src/graph.py` (539 lines) — LangGraph assembly

Responsibility: Defines `GraphState` `TypedDict` + 9 node functions + conditional routing. Hosts the D3 policy gate node and auto-approve shim as additive additions.

I/O: `build_graph()` $\rightarrow$ `StateGraph` / `compile_graph()` for the legacy/replay path.

Touched layers: All three. Operations/Cost nodes $\leftrightarrow$ operational layer; Governance $\leftrightarrow$ output contract; Supervisor $\leftrightarrow$ supervision layer; Evaluation $\leftrightarrow$ evaluation layer.

Key contracts:

- `policy_gate_node` is **opt-in** via `state["policy_gate_enabled"]=True`. Default-False path preserves V2 behavior exactly (supervisor always runs, evaluation always runs).
- When opt-in, `_policy_gate_route` routes: `AUTO_EXECUTE` $\rightarrow$ `auto_approve_shim` $\rightarrow$ `evaluation`, `HUMAN_REQUIRED` $\rightarrow$ `supervisor` $\rightarrow$ `evaluation`.
- `auto_approve_shim` synthesizes a minimal `supervisor_output` dict (shape-compatible with `SupervisorDecision.to_dict()`) without importing `supervisor.py` — lets `evaluation_node` run on auto-executed cases in the legacy path.

Risk points: The `auto_approve_shim` is a legacy-path convenience. In the D3 runtime path (`event_loop.py`), the shim is NOT used — `evaluation_node` is bypassed entirely, which is correct runtime/research-core separation.

Maturity: Stable. `test_week2_contracts.py` (15) + `test_policy_gate.py` integration tests cover topology.

`src/event_loop.py` (399 lines) — D3 outer orchestrator

Responsibility: The D3 runtime entry point. Iterates an event list, patches state, runs inner reasoning slice, handles routing, dispatches adapter or skips, accumulates outcomes, injects summaries.

I/O:

```
run_event_loop(
    events: list[EventPayload],
    initial_twin_state: TwinState | None = None,
    outcome_store: OutcomeStore | None = None,
    auto_approve_escalations: bool = False,
) -> dict # event_results + events_processed + outcomes_count + outcome_summary + final_twin_state_summary
```

Touched layers: Operational + policy-routing only. Never evaluation.

Key contracts:

- `outcome_summary` is injected into `scenario_context` **before** current event's reasoning — reflects prior outcomes only, never self-references.
- `HUMAN_REQUIRED` defaults to skip (`execution_status="awaiting_human_review"`), never silently auto-executes.
- Failed execution (infeasible or parse error) converts to `execution_status="execution_failed"` with `execution_error` string; `outcome_store` unchanged.
- Unknown route → `execution_status="unknown_route"` + diagnostic error.

Risk points: No persistence (losing state if process dies). No concurrency. Inner reasoning runs sequentially; LLM-mode agents would make each event O(seconds). Acceptable for Path B demo.

Maturity: Mature. 23 tests (Phase 2 + Phase 3 additive).

`src/execution_adapters.py` (526 lines) — Action executors

Responsibility: 4 deterministic adapters + metadata-first dispatcher. Produces mutated `TwinState` + `ExecutionOutcome`.

I/O:

- `execute_expedite` / `execute_transfer` / `execute_compensate` / `execute_no_action`: each takes `TwinState` + kwargs `event_id`, `timestamp`, `governance_meta` → returns `(TwinState, ExecutionOutcome)`
- `execute_action(governance_output, twin_state, event_id, governance_meta, timestamp)` — unified dispatcher

Touched layers: Operational only. Explicitly rejects APPROVE/VERIFY/OVERRIDE and AI/ALT1/ALT2 in both metadata-first and text-fallback parsing.

Key contracts:

- Deep-copies `TwinState`; never mutates input.
- Reuses `TwinState.apply_action` + `compute_total_cost` — no parallel cost engine.

- EXPEDITE/TRANSFER raise `ExecutionInfeasibleError` on state-infeasibility (no alternate carrier / source empty / etc.) — fail-closed.
- `_parse_operational_type` returns `(action_type, parse_source)` tuple; dispatcher annotates `outcome.notes` with `[parse=text_fallback]` when natural-language fallback was used.
- Post-Phase-3.5: unknown non-empty metadata raises, not falls through silently.

Risk points:

- `_pick_alternate_carrier` uses `cid` in `hint_id` substring — benign today (CR_1/CR_2 distinct) but fragile if more carriers are added (e.g., CR_10 would substring-match CR_1).
- Adapter still depends on `governance_output.recommended_action` text format staying `"{OPERATIONAL_TYPE}:..."`; agent schema drift would break parsing.

Maturity: High. 23 tests + 19 hardening-specific locks.

src/outcome_store.py (159 lines) — Append-only audit

Responsibility: In-memory, append-only store. Tracks outcomes + routing decisions separately.

I/O:

- `append(outcome: ExecutionOutcome)` — strict type check
- `record_routing(route: str)` — counts routing decisions even when no outcome is produced (HUMAN_REQUIRED skips)
- `all()` / `count()` / `latest()`
- `summary(recent_n=5)` → 11-field deterministic aggregates

Touched layers: Operational outcomes + routing counts. No evaluation semantics.

Key contracts: No delete/update/clear/pop/remove methods (verified by `test_no_delete_method`). Summary fields stable: `outcomes_count`, `total_cost`, `avg_cost`, `action_counts` (4 op types), `status_counts` (3 statuses), `sla_preserved_count`, `sla_missed_count`, `auto_executed_count`, `human_required_count`, `recent_actions` ($\leq N$ compact), `recent_resolution_patterns` (`action_type:status` histogram).

Risk points: No persistence — each `event_loop` run starts fresh unless caller supplies a store. No summary schema versioning (Phase 4 should not break the contract).

Maturity: High. 15 tests.

src/policy_gate.py (184 lines) — Deterministic routing

Responsibility: Given `governance_output["risk_level"]` and `PolicyGateConfig`, returns a `PolicyDecision`.

I/O: `decide_policy(governance_output, config=None) → PolicyDecision`

Touched layers: Only reads `risk_level` from governance output. Never LLM text, never cost magnitude, never confidence_note.

Key contracts:

- Fail-closed: unknown/missing/empty `risk_level` → `HUMAN_REQUIRED` with `RiskLevel.HIGH` conservative fallback.
- `PolicyGateConfig` requires `auto_levels` $\cup$ `human_required_levels` = {LOW, MEDIUM, HIGH} and disjoint.
- Default config: `LOW→AUTO`, `MEDIUM/HIGH→HUMAN`.
- Case-insensitive + whitespace-tolerant `risk_level` parsing.

Risk points: None at current scope. Cannot be contaminated by LLM-derived fields because it only reads one whitelisted field.

Maturity: Very high. 18 tests including fail-closed + config validation.

`src/evaluation.py` (328 lines) + `src/action_code_mapper.py` (205 lines) — Research Core sidecar

Responsibility:

- `evaluation.py`: Deterministic regret / override / cost functions reading oracle case JSON (`oracle.action_code`, `oracle.cost_total`, `cost_ground_truth` per action).
- `action_code_mapper.py`: The only sanctioned bridge between supervision layer (APPROVE/VERIFY/OVERRIDE) and evaluation layer (AI/ALT1/ALT2).

I/O:

- `evaluation`: `get_total_cost` / `compute_total_cost` / `is_override` / `is_unnecessary_override` / `compute_override_effectiveness` / `compute_regret` / `validate_split_cost` / `validate_oracle_consistency` / `get_agent_action_code` / `normalize_action_code`
- `action_code_mapper`: `map_supervisor_to_action_code(supervisor_output, case_data) → ActionCode`

Touched layers: Evaluation layer + supervision layer (via the mapper).

Key contracts:

- **Frozen.** Never modified. Never called from D3 runtime path.
- Evaluation truth comes exclusively from pre-computed case JSONs, not from runtime outcomes.
- `action_code_mapper.py` imports `evaluation.normalize_action_code` — this is the one cross-module import inside the sidecar.

Risk points: Zero — by construction, no runtime D3 module imports either. Import graph verified: only `graph.evaluation_node` imports these, and that node is not on the D3 event loop's inner reasoning slice.

Maturity: Very high. `test_evaluation.py` (76) + `test_evaluation_node.py` (12) + `test_action_code_mapper.py` (20) + `test_phase6_batch_eval.py` (13).

Other V2 modules (briefly)

- `supervisor.py` (345): Supervision decisions (APPROVE/VERIFY/OVERRIDE). Only invoked on legacy graph path when policy gate routes to HUMAN_REQUIRED. 48 tests.
 - `agents/operations_agent.py` (1013): Candidate generation + feasibility gating. Emits EXPEDITE/TRANSFER/COMPENSATE/NO_ACTION. 49 tests.
 - `agents/cost_agent.py` (476): Runtime cost estimates per candidate (not evaluation truth). 28 tests.
 - `agents/governance_agent.py` (930): 8-field structured output synthesis + risk_level determination. 23+15 tests.
 - `rag_setup.py` / `retrieval.py` (2117 total): Vector store, retrieval strategies (tfidf_legacy/dense/hybrid/hybrid_rerank). ~88 tests across 5 retrieval files.
 - `llm_backend.py` + `llm_providers/*` (716): Optional LLM gateway with fallback chain (anthropic + openai). 24 tests.
-

6. Critical Contracts (with file references)

6.1 Event contract — `event_schema.py`

- `EventType`: {CARRIER_DELAY_ESCALATION, WEATHER_WORSENING, DEMAND_SPIKE, INVENTORY_DISCREPANCY, CUSTOMER_CANCELLATION, COMPLIANCE_HOLD} — closed, 6 types.
- `EventSeverity`: {LOW, MEDIUM, HIGH}.
- `EventPayload`: immutable (frozen), tz-aware timestamp required, non-empty `event_id`, `parameters`: dict as free bag but no per-type discriminator yet.
- `AffectedEntityRef`: typed entity reference with non-empty `entity_type` + `entity_id`.

6.2 Outcome contract — `outcome_schema.py`

- `ExecutionOutcome`: frozen, `action_taken`: Literal["EXPEDITE", "TRANSFER", "COMPENSATE", "NO_ACTION"] — operational layer only.
- Pydantic validator explicitly rejects APPROVE/VERIFY/OVERRIDE and AI/ALT1/ALT2 on `action_taken`.
- `cost_incurred` ≥ 0 enforced.
- `status`: Literal["EXECUTED", "SKIPPED", "REJECTED"].

6.3 Policy gate contract — `policy_gate.py`

- Single input: `governance_output["risk_level"]` . All other fields explicitly ignored for routing.
- Output: `PolicyDecision(route, risk_level, requires_human_review, reason, policy_config_version)` .
- Fail-closed on missing/unknown.
- Config must cover {LOW, MEDIUM, HIGH} disjointly.

6.4 Mutation discipline — `twin_state.py` + `execution_adapters.py`

Three entry points, all atomic, no fourth style:

1. `inject_shock(shock_config)` — V2 scenario injection, via `_run_patches` .
2. `apply_event_patch(event)` — D3 event hot-patch, via `_run_patches` with event-type → patch mapping.
3. `apply_action(action)` — V2 operational action, internal Pydantic `validate_assignment` .
4. `apply_action_outcome(outcome)` — REPLAY-ONLY narrow utility. Uses `_run_patches` on `state_delta["patches"]` . Set-only patches ensure idempotency.

6.5 Outcome summary injection contract — `event_loop.py:148`

- Injected into `scenario_context["outcome_summary"]` **before** inner reasoning runs on the current event.
- Reflects outcomes 1..N-1 only (never self-references event N).
- GovernanceOutput 8-field schema is **NEVER extended**. Summary stays as sideband context.
- On first event, summary reports `outcomes_count=0` (empty-but-valid summary).

6.6 GovernanceOutput boundary — `agents/governance_agent.py`

- Exactly 8 frozen fields: `risk_level`, `recommended_action`, `situational_explanation`, `cost_summary`, `confidence_note`, `rationale_trace`, `evidence_sources[]`, `alternative_actions[]` .
- Identity metadata (`recommended_candidate_id`, `recommended_candidate_type`, `alternatives[]`) lives in the **separate** `_governance_meta` state field, not inside GovernanceOutput.
- D3 never extends the schema; outcome_summary is strictly sideband.

6.7 Layer separation — enforced across code

- **Operational**: EXPEDITE / TRANSFER / COMPENSATE / NO_ACTION. Used by ops agent, cost agent, adapters, outcome schema.

- **Supervision:** APPROVE / VERIFY / OVERRIDE. Used by supervisor, policy gate in routing context, `auto_approve_shim`.
- **Evaluation:** AI / ALT1 / ALT2. Used by `evaluation.py`, `action_code_mapper.py`, oracle case JSONs.
- `action_code_mapper.map_supervisor_to_action_code` is the **only** bridge.
- `execution_adapters.py` explicitly rejects non-operational prefixes at parse time (Phase 3.5 hardening).

6.8 Truth boundary — documented + enforced by import topology

Verified via grep: NO D3 runtime module (`event_loop`, `event_state_mapper`, `execution_adapters`, `outcome_store`, `event_engine`, `event_schema`, `outcome_schema`, `event_sequence`, `twin_state`, `policy_gate`) imports `evaluation` or `action_code_mapper`. Only `graph.py` imports them (inside `evaluation_node`), and that node is bypassed in the D3 inner reasoning slice. `evaluation_node` reads `data/cases/*.json` — the only runtime file that does.

7. Actual End-to-End Flow (single event walkthrough)

Setup: `run_event_loop([e1, e2, e3])`, `initial_twin_state=None` (baseline loaded), `outcome_store=None` (fresh store), `auto_approve_escalations=False`.

Event 1: EVT-D01 (CARRIER_DELAY_ESCALATION, severity=LOW, CR_1 +6h)

1. **Event arrives** — `EventPayload` read from list.
Module: `caller / event_engine.generate_demo_event_stream`.
2. **State patch** — `map_event_to_state(e1, live_state)` deep-copies baseline, calls `patched.apply_event_patch(e1)` which computes `patches=[{entity:carrier, id:CR_1, op:add, field:transit_time_hours, value:6.0}]`, applies via `_run_patches` with draft+commit, appends `{source:event_patch, event_id, ...}` to `current_disruptions`. Returns `(patched, scenario_context)`. `live_state ← patched`.
Modules: `event_state_mapper.py`, `twin_state.apply_event_patch`.
3. **Summary injection** — `scenario_context["outcome_summary"] = outcome_store.summary()`. Store is empty, so summary has `outcomes_count=0`, empty `recent_actions`, zero counters.
Module: `event_loop.run_event_loop` line 148.
4. **Reasoning** — `_run_reasoning_slice(twin_dict, scenario_context)` calls `operations_agent_node → cost_agent_node → governance_agent_node → policy_gate_node` sequentially on a fresh `GraphState` dict with `policy_gate_enabled=True`.
 - **ops:** ranks EXPEDITE/TRANSFER/COMPENSATE/NO_ACTION candidates using the patched twin + retrieval context.
 - **cost:** runtime cost estimates per feasible candidate (not evaluation truth).
 - **governance:** synthesizes 8-field output; `risk_level` determined as LOW (severity=LOW, baseline fully feasible); `recommended_action` is formatted as `"{TYPE}: ..."`; `_governance_meta` includes `recommended_candidate_type`.

- **policy_gate**: reads `risk_level="LOW"`, returns `PolicyDecision(route=AUTO_EXECUTE, ...)`.

Modules: `graph.py` (4 node functions), `agents/*`, `policy_gate.py`.

5. **Routing + execution** — `_route_and_execute("AUTO_EXECUTE", ...)` calls `record_routing("AUTO_EXECUTE")` then `_try_execute(...)`:

- `execute_action(gov, live_state, event_id="EVT-D01", governance_meta, timestamp) → _parse_operational_type` returns `("COMPENSATE", "metadata")` (metadata path, in practice based on my end-to-end smoke) → dispatches `execute_compensate`.
- `execute_compensate` deep-copies state, builds an `Action(COMPENSATE, units=order_units)`, calls `apply_action + compute_total_cost` → returns `(new_state, outcome)` where `outcome.action_taken="COMPENSATE"`, `cost_incurred ≈ 460.00`, `state_delta.patches=[]` (compensation doesn't touch physical state), `status="EXECUTED"`, `notes="Compensation applied..."`.
- `outcome_store.append(outcome)`, `live_state ← new_state`, returns `("executed", outcome.model_dump(), None, new_state)`.

Modules: `event_loop._try_execute`, `execution_adapters.execute_action + execute_compensate`, `twin_state.apply_action + compute_total_cost`, `outcome_store.append`.

6. **Record collection** — one `event_results` entry: `{event_id, event_type, severity, scenario_context (includes outcome_summary), governance_output (8 fields), policy_decision (route, risk_level, reason), execution_status="executed", execution_outcome={...}, trace_log}`.

7. **Next event** — `EVT-D02 (WEATHER_WORSENING, MEDIUM)`. `outcome_store.summary()` now has `outcomes_count=1`, `action_counts.COMPENSATE=1`, `total_cost=460.00`, ... This summary is injected into `e2's scenario_context["outcome_summary"]` before `e2's` reasoning. Governance produces `risk_level=MEDIUM`; policy gate → `HUMAN_REQUIRED`; `record_routing("HUMAN_REQUIRED")`; `execution_status="awaiting_human_review"`; `execution_outcome=None`; live state advances only via event patch, no adapter execution.

8. Current Capability Inventory

What works now

- Deterministic synthetic event generation (6 types, fixed IDs, ISO-8601 timestamps).
- Atomic event hot-patching on live `TwinState` across all 6 event types.
- Policy-gated routing with fail-closed semantics.
- Four fully-deterministic execution adapters grounded in V2 cost logic.
- Append-only in-memory outcome store with 11-field summary aggregates.
- Outcome summary injection into subsequent reasoning context (sideband only).
- Legacy path (compiled graph + supervisor + `evaluation_node`) still fully operational for V2 regression and Streamlit demo.
- Research Core evaluation runs unchanged on 20 oracle cases.

- Phase 1 `event_sequence.run_case_sequence` fallback runner for case-id-list replay.

What is demoable now

- **Headless CLI demo:** `run_event_loop(generate_demo_event_stream())` prints a 6-event walkthrough showing routing/execution/skip/state accumulation (demonstrated live during Phase 3).
- **V2 Streamlit case viewer** (`app.py`): selects a case, runs it through the compiled graph with policy gate enabled, displays scenario / twin state / governance / supervisor / evaluation panels + D3 policy-gate pill (auto-execute ⚡ vs human 👤).
- **Research Core:** batch evaluation via `scripts/phase6_batch_eval.py`.

What is robust now

- Contract discipline (schemas, enums, fail-closed).
- Deterministic behavior (no randomness, no wall-clock dependencies except configurable timestamp).
- Test suite: 1040/1040 pass, covering every module's public surface.
- Atomic mutation with rollback on any patch failure.
- Truth-boundary isolation (verified by import graph).
- Parse-source audit annotation (metadata vs text_fallback).
- Demo-override audit annotation.

What is still fragile

- **Agent behavior contract drift:** adapters depend on `recommended_action` being `"{OPERATIONAL_TYPE}: ..."` format; any agent change would degrade to text-fallback or `ExecutionInfeasibleError`.
- `_pick_alternate_carrier` **substring matching:** works for CR_1/CR_2 but not for carrier IDs with shared prefixes/suffixes.
- **No persistence:** each `event_loop` run is a fresh process; no replay across sessions.
- `scenario_context["outcome_summary"]` **is plumbed but not consumed:** agents don't yet demonstrably change behavior based on injected history — so the loop closure is structural, not behavioral.
- **Streamlit surface is V2 case-centric**, not event-centric (no event timeline, no outcome log panel).
- **No concurrency model** — sequential by design.

What is intentionally deferred

- Phase 4 (event-centric UI, React option).
- Path C: learning module, adaptive policy thresholds, replan trigger, event correlator, session management, multi-run analytics.

- Real outcome physics (current = `TwinState.compute_total_cost`, which is V2 deterministic but coarse).
- Schema versioning on `outcome_summary`.
- External persistence.
- Real human approval workspace.

9. Test and Reliability Picture

Coverage breakdown (1040 tests across 31 files)

Layer	Files	Tests	Density
V2 agents (ops/cost/gov + LLM mode)	5	49+28+23+23+15 = 138	High
V2 twin_state / scenarios	2	25+54 = 79	High
V2 retrieval / RAG	5	20+23+17+20+15+13 = 108	Very high
V2 evaluation / mapper / case-gen	4	76+20+351+10 = 457	Extreme (case-gen is property-style)
V2 graph + supervisor + api	3	15+48+20 = 83	High
V2 LLM + phase6	3	21+3+13 = 37	Medium
D3 schemas + contracts	2	13+15 = 28	Adequate
D3 policy gate	1	18	High
D3 event loop + execution + outcome store	3	23+23+15 = 61	High
D3 hardening locks	1	19	Focused
D3 integration (evaluation_node, event_state_mapper)	2	12+15 = 27	Adequate

Strongest coverage areas

- Evaluation math (76 + 351 case-gen permutations).
- Retrieval (88 tests across 5 files, all 4 retrieval modes).
- Operations agent (49 unit + 23 llm-mode + 15 governance synthesis integration).
- Policy gate (18 incl. fail-closed matrix + config validation).

- Atomic mutation (inject_shock rollback tests, apply_event_patch atomicity, apply_action_outcome atomicity).

Blind spots

- **Outcome summary as actual context signal:** no test asserts that a non-empty outcome_summary causes measurable agent behavior change. (By design — behavior change is Path C.)
- **Cross-event compounding on state:** only 2 tests verify state accumulation across 4+ events; stress scenarios with 20+ events not tested.
- **Cross-event COMPLIANCE_HOLD + subsequent EXPEDITE infeasibility cascading:** event_loop doesn't have a targeted test for "hold CR_2 then try to expedite" in the event path (adapter-level test exists).
- **Human-required routing path behavior under real supervisor instructions:** the D3 runtime skips supervisor entirely; only legacy path exercises it. No integration test covers "policy gate says HUMAN_REQUIRED → supervisor VERIFY → evaluation" through the D3 path (because D3 path doesn't flow through supervisor by design).
- **FastAPI endpoints** (`src/api/*`): 20 tests but not clearly aligned with D3 main path.

Fit for UI layer?

Yes, with caveats. Contracts are stable, outputs are structured JSON-serializable, audit annotations are in place. UI consumers should bind to:

- `event_results[*].execution_status` (5 enum values)
- `event_results[*].execution_outcome` (dict or null)
- `event_results[*].execution_error` (string, optional)
- `event_results[*].policy_decision.route + .risk_level + .reason`
- `outcome_summary.*` (11 fixed fields)

Fit for Path C core?

Partially. Summary injection channel exists; the physical plumbing (inject before reasoning, never self-reference) is tested and locked. But to add *behavioral* learning, you'd need:

- Agent prompts that actually reference `scenario_context["outcome_summary"]`.
 - A stable outcome_summary schema (currently has no version; Phase 4 or C risks breaking consumers).
 - A policy-gate override module separate from the current static `PolicyGateConfig`.
- None of these exist yet. Before Path C, these contracts should be frozen into a `D3_BASELINE_v1` snapshot.
-

10. Architectural Strengths

1. **Additive-only discipline is visibly preserved.** V2 agents, supervisor, evaluation, and retrieval are 100% unchanged in code. Diff-ability between V2 and D3 is clear.
 2. **Truth boundary is not just documented — it's enforced by import topology.** Grepping the D3 runtime path confirms zero imports of `evaluation.py` or `action_code_mapper.py`.
 3. **Single mutation discipline.** `_run_patches` is the one patch engine; all three entry points route through it. No third mutation style snuck in during Phase 3.
 4. **Outer orchestrator pattern.** The D3 event loop is a Python `for` loop wrapping direct calls to V2 node functions. The LangGraph graph was never rewritten to self-cycle — a major design risk (per roadmap §12) successfully avoided.
 5. **Fail-closed posture.** Unknown risk levels → `HUMAN_REQUIRED`. Unknown route → `execution_failed`. Missing metadata + no text → `GovernanceActionParseError`. Infeasible execution → `ExecutionInfeasibleError`. No silent degradation.
 6. **Layer separation by type system.** `ExecutionOutcome.action_taken` is `Literal[...]` of operational identifiers only, with Pydantic validators rejecting supervision/evaluation tokens. It's impossible to store cross-layer contamination in the outcome store.
 7. **Opt-in policy gate.** `policy_gate_enabled=False` default preserves V2 graph behavior bit-for-bit. No V2 test broke when D3 nodes were added to the graph.
 8. **Audit annotations (Phase 3.5).** `[parse=text_fallback]` and `[DEMO_OVERRIDE on HUMAN_REQUIRED]` markers in `outcome.notes` make structural audit possible without additional fields or side channels.
 9. **Research Core frozenness is load-bearing.** The 20 oracle cases + evaluation metrics remain trustworthy for thesis claims, even as runtime evolves.
-

11. Architectural Weaknesses / Technical Debt

Harmless debt (can defer indefinitely)

- ~5 `.DS_Store` files and ~10 `__pycache__` directories still in working tree. `.gitignore` now prevents future pollution but existing tracked noise wasn't auto-cleaned per Phase 3.5 scope.
- Two Office temp lock files `~$_Demo_Implementation_Roadmap.docx` / `~$_Demo_Path_C_Vision.docx` (Word lock artifacts — visible in scan, not worth git-ignoring).
- `src/api/*` FastAPI stack is untouched but not clearly positioned — neither used nor deleted. Adds to surface area without current purpose.
- `src/case_translation.py` and `scripts/*` are V2 batch utilities with no D3 runtime role.

Real debt (must-fix before Path C)

- **outcome_summary has no schema version.** Consumers that bind to the 11 fields today can silently break if Phase 4/C adds fields. Before Path C, add a `schema_version: "v0"` field or lock it as `D3_BASELINE_v1`.
- **No persistence for OutcomeStore.** Learning requires cross-session memory; in-memory store can't support it. Path C will need to decide: pickle / JSON-lines append / SQLite.
- **Agent prompts don't consume outcome_summary.** The channel is built but not exercised. Path C must either extend prompts (violating "agents frozen") or introduce a new agent layer that reads the summary.
- **_pick_alternate_carrier substring matching.** Benign today, breaks if IDs become e.g. `CR_1`, `CR_10`, `CR_100`. Switch to explicit equality before any network-size change.
- **Text-fallback dependency on agent output format.** `execute_action` still falls back to parsing `recommended_action` prefix. If `governance_agent` changes output format, silent degradation to `execution_failed`. Consider making `_governance_meta.recommended_candidate_type` mandatory (and raise cleanly if missing) — Path C should formalize this.

Blocker-class debt if undiagnosed

- **Streamlit app.py is entirely V2 case-centric.** Phase 4 either extends it (risking regressions to the V2 legacy demo) or replaces it (breaking V2 demo behavior). D3 needs a decision: parallel Streamlit page vs replacement. Current `app.py` already has a D3 policy-gate pill injected additively; this pattern can scale to event-centric.
- **No health check / contract-snapshot test.** No single test asserts "the `outcome_summary` output of `run_event_loop(demo_stream())` matches a checked-in JSON snapshot". Without this, schema drift can pass CI.

12. Path C Readiness Assessment

Is the starting point adequate?

Yes, provisionally, after a contract-freeze step. The event loop is stable, mutation discipline is single-source, `outcome_store` is append-only and deterministic, policy gate is pure and fail-closed. These are exactly the properties a learning/adaptive layer needs to safely build on.

What can be reused as-is

- `event_schema.EventPayload` + `event_engine` (all 6 types + factory helpers + demo stream).
- `event_state_mapper.map_event_to_state` (pure function).
- `twin_state.apply_event_patch` + `apply_action_outcome`.
- `execute_action` dispatcher and 4 adapters.

- `outcome_store.OutcomeStore` (append-only is exactly the episodic-memory precondition).
- Policy gate (for adaptive: wrap, don't replace).
- Outcome summary injection mechanism.

Contracts that must be frozen into `D3_BASELINE_v1` before Path C

1. `outcome_summary` 11-field shape (add version tag).
2. `event_result` record shape (9 top-level keys).
3. `policy_decision` shape (5 fields).
4. `ExecutionOutcome` 8-field schema.
5. `EventPayload` 6-field schema.
6. `_governance_meta` dict shape (recommended_candidate_id, recommended_candidate_type, alternatives).
7. Adapter return contract (TwinState, ExecutionOutcome).
8. Truth-boundary import rules (codified as an import-linter check).

Natural Path C extension points

- **`src/learning_module.py` (NEW):** reads `outcome_store.all()`, returns structured query interface (by `event_type` / `risk_level` / `action_type`). Writes summaries back into `scenario_context` via a richer injection than the current statistical aggregate.
- **`src/replan_trigger.py` (NEW):** a wrapper around `event_loop` that inspects `execution_outcome.cost_incurred` vs expected cost (from `governance.cost_summary`) and emits a `replan` event. Requires adding an inner inner-loop — this is where the outer-orchestrator pattern starts to strain, and may justify a state-machine wrapper.
- **`src/event_correlator.py` (NEW):** sliding window over the last N events, emits `compound_event` alerts as a synthetic new event into the queue. Reuses `EventPayload` schema.
- **`src/policy_gate.py` (EXTEND):** adaptive thresholds. Add a new `AdaptivePolicyConfig` subclass that can re-derive `auto_execute_levels` per `event_type` based on `outcome_store.summary()`. Keep `PolicyGateConfig` unchanged for determinism baseline.
- **`app.py` / new `src/ui/event_dashboard.py`:** Phase 4 event timeline + decision panel + outcome log. Must consume the frozen v1 contract.

Absolutely do not directly modify

- `src/evaluation.py` / `src/action_code_mapper.py` / `data/cases/*.json` — Research Core.
- `V2_agents/*` — any change breaks deterministic reasoning reproducibility against V2 baseline.
- `D3_BOUNDARY.md` — change only by explicit governance decision with updated version.
- `src/policy_gate.decide_policy` body — extend via new config class, not by changing core routing.

Readiness verdict

Green for contract-freeze + Phase 4 UI. Yellow for Path C core — needs schema versioning + a persistence decision + a clear "where do adaptive thresholds live" decision before Learning module gets wired up.

13. Recommended Next Step

Recommendation: Proceed to Phase 4 (event-centric UI) before Path C.

Reasoning

Arguments for Phase 4 first:

- The backend is contractually stable (1040 tests green including 19 Phase 3.5 hardening locks).
- A visible event loop is essential for Path C demos too — Path C success criteria include "non-technical user can operate the system" and "session replay". UI work is a prerequisite, not a parallel track.
- Phase 4 consumes contracts without modifying them. It's the least-risky way to validate the contract shapes are actually consumable before freezing them into `D3_BASELINE_v1`.
- Building Path C on top of an untested contract (no UI has ever read `outcome_summary`) would be premature — Phase 4 implicitly contract-tests the output by displaying it.

Arguments against starting Path C directly:

- Path C's first real feature (adaptive thresholds or replan) would require either breaking the `policy_gate.decide_policy` signature or adding a new config hierarchy — a decision that benefits from usage signal which only Phase 4 provides.
- `Outcome_summary` is plumbed but not behaviorally consumed. Path C learning work on an un-proven channel risks building atop a contract that's about to change.
- Path C's per-phase effort estimate (~80h beyond Path B) is larger than Phase 4's (~10h). Completing Phase 4 crystallizes whether the scope-cut ladder (drop UI, fall back to Phase 1-3) was ever needed.

Caveats if you choose Phase 4 next

- Build event-centric UI as an **additional** Streamlit page (e.g. `app_events.py` or a tab), not a replacement. Preserve the V2 case-centric page to keep the legacy demo intact.

- Bind UI strictly to the documented contract (§6.5 and §6.4 of this report). Do NOT read `_governance_meta` or undocumented internal fields from UI — treat the `event_result` record as the stable surface.
- Snapshot-test the output of `run_event_loop(generate_demo_event_stream())` into a JSON fixture under `tests/fixtures/` and add a test comparing current output to that fixture. This closes the one critical test blind spot identified in §9.
- Do not modify `event_loop` / `adapters` / `policy_gate` during UI work. If UI wants a richer record, that's either a `stable-v1` schema bump or a UI-only adapter layer.

Caveats if you instead choose Path C next

- First action must be: freeze `D3_BASELINE_v1` contracts (write a schema fixture + snapshot test) before any learning module code.
- Second action: decide persistence format for OutcomeStore (JSONL vs pickle vs SQLite). Recommendation: JSONL append, human-readable, schema-agnostic.
- Third action: keep V2 + Phase B tests green throughout. If any Phase B test needs to change, you've violated additive-only.

Appendices

Appendix A — Frozen reuse list (do not modify)

File	Type	Rationale
src/evaluation.py	Research Core	Thesis-grade oracle truth
src/action_code_mapper.py	Research Core	Three-layer sole bridge
data/cases/*.json (20 files)	Research Core	Frozen oracle data
data/scenarios/*.json (7 files)	V2 scenario bank	Used by case generation
data/knowledge/*.md (5 files)	RAG corpus	Retrieval grounding
src/agents/operations_agent.py	V2 reasoning	Candidate generation
src/agents/cost_agent.py	V2 reasoning	Runtime cost estimation
src/agents/governance_agent.py	V2 reasoning	8-field frozen schema
src/supervisor.py	V2 supervision	Legacy path only
src/rag_setup.py + src/retrieval.py	V2 retrieval	Reused unchanged

File	Type	Rationale
src/llm_backend.py + src/llm_providers/*	V2 LLM gateway	Reused unchanged
src/event_schema.py	D3 Phase 0	Event contract immutable
src/outcome_schema.py	D3 Phase 0	Outcome contract immutable
src/policy_gate.py	D3 Phase 1	Routing core; extend via new config class
D3_BOUNDARY.md	Governance doc	Source of truth
SPEC.md / SPEC_BRIDGE.md	V2 governance	Frozen historical spec
All existing test files (31)	Regression safety net	Must stay green

Appendix B — Additive extension list (may extend, never break)

File	Allowed extensions
src/twin_state.py	Add new mutation methods via <code>_run_patches</code> ; never change existing method signatures
src/graph.py	Add new nodes/edges additively; never modify existing node bodies
src/event_engine.py	Add new event factories or streams; 6 event types enum stays closed
src/event_state_mapper.py	Add <code>scenario_context</code> fields; don't remove existing ones
src/event_loop.py	Add new kwargs with safe defaults; don't break current signature
src/execution_adapters.py	Add new helpers / tighten parsing; adapter public signatures stay
src/outcome_store.py	Add summary fields (with version bump) or new query methods; never delete fields
app.py	Add event-centric sections additively; preserve V2 case viewer
tests/test_*.py (existing)	Add new test cases; don't rewrite existing assertions

Appendix C — High-risk files list (changes require extra care)

File	Risk class	Why
src/twin_state.py	Highest	3 mutation APIs share <code>_run_patches</code> ; any change touches all three. Pydantic validators enforce invariants downstream. 864 lines.

File	Risk class	Why
src/graph.py	High	D3 runtime and legacy path both use node functions directly. <code>auto_approve_shim</code> is a subtle legacy-only detail. 539 lines, 9 nodes, conditional routing.
src/execution_adapters.py	High	Consumes governance output in two parsing modes; rejects two non-operational layers; couples to <code>twin_state.apply_action</code> semantics. 526 lines.
src/event_loop.py	Medium-high	Outer orchestrator where outcome summary timing, routing, and adapter dispatch all converge. Breaking any order assumption creates subtle contamination bugs. 399 lines.
src/agents/governance_agent.py	Medium-high (frozen)	8-field schema is load-bearing; <code>risk_level</code> determination logic feeds the policy gate. 930 lines — don't touch without schema version bump.
src/policy_gate.py	Medium	Small but load-bearing. Any change here flips every HUMAN_REQUIRED demo case and must propagate to PolicyDecision consumers. 184 lines.
src/evaluation.py	Medium (frozen)	Mathematical correctness of regret / override metrics depends on this. 328 lines.
src/outcome_store.py	Medium	Append-only contract is a load-bearing simplification; any delete/update API addition breaks learning-module assumptions later. 159 lines.

End of report.